

PRACTICAL 1

Deploy Nginx using Docker and Kubernetes

A hands-on lab for learning Docker containers and Kubernetes Deployments, Pods, and Services.

Student details

Field	Details
Name	SHWETA SANDESH UBALE
Roll No. / ID	MCA2504150
Batch / Section	SECTION B
Date performed	15/092026

Learning objectives

By the end of this practical, you should be able to explain and demonstrate:

- What a Kubernetes cluster is
- What a Node is
- What a Pod is
- What a Deployment is
- What a Service is
- How Kubernetes uses Docker / container images
- How to deploy an application on Kubernetes
- How to scale an application
- How to check application logs
- How to delete Kubernetes resources

Before you start

Don't start with Kubernetes YAML, AKS, Helm, Ingress, Terraform, or Jenkins yet. This first practical is done entirely with docker and kubectl commands. Practical 2 will build your own image, push it to Docker Hub, and deploy it with a YAML Deployment and Service — that's where Docker and Kubernetes really connect.

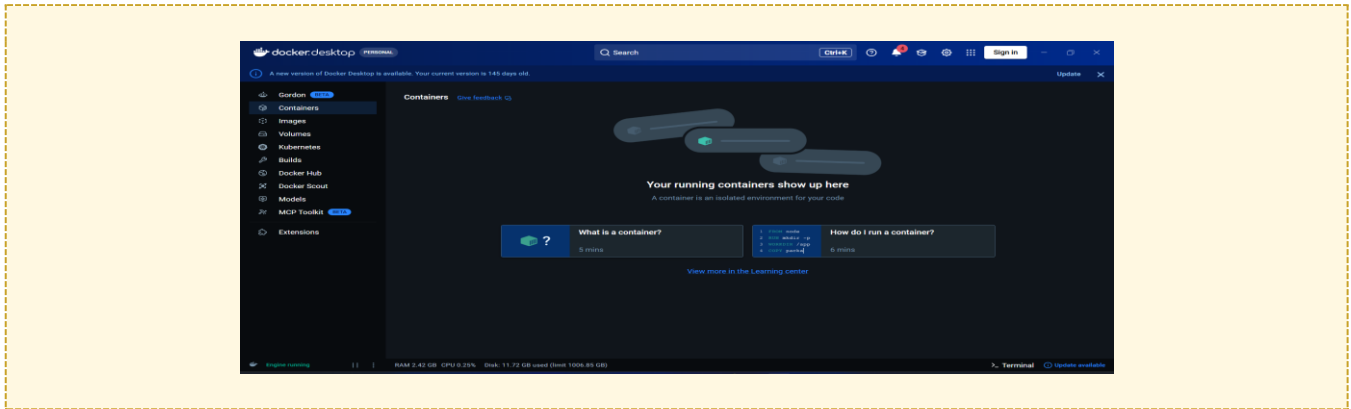
Contents

(Right-click below and choose "Update Field" in Word to generate page numbers.)

Part A — Check Docker

Step A1: Start Docker Desktop

Open Docker Desktop and make sure it is running (the whale icon should be steady, not animating).



Step A2: Check the Docker version

Open Command Prompt or PowerShell and check that Docker is installed correctly.

Run:

```
docker --version
```

Expected output:

```
C:\>docker --version
Docker version 29.4.1, build 055a478
```

Step A3: Check running containers

It is okay if the list is empty — you just want to confirm the command works.

Run:

```
docker ps
```

Expected output:

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
--------------	-------	---------	---------	--------	-------	-------

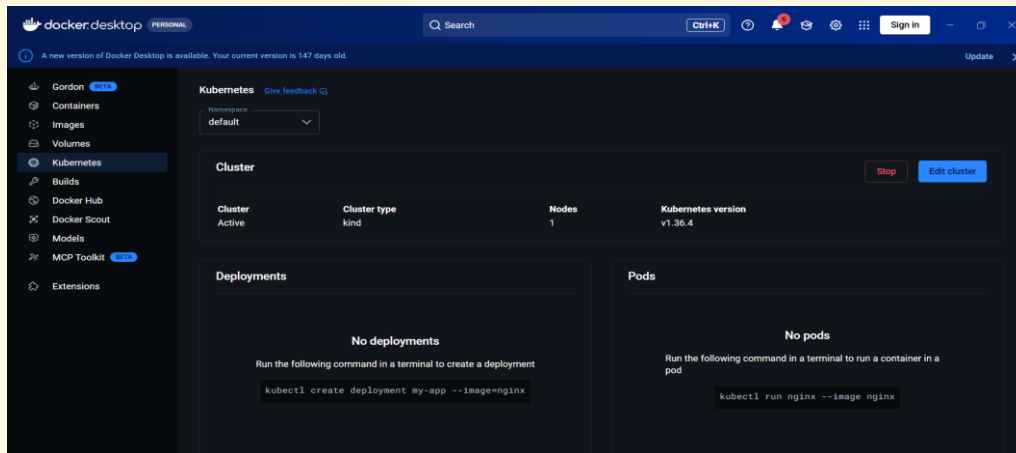
```
C:\>docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
f11dd2fd14f8   docker/desktop-cloud-provider-kind:v0.5.0   "/bin/cloud-provider..."  5 minutes ago   Up 5 minutes   2375-2376/tcp
kind-cloud-provider
4651c2109cad   docker/desktop-containerd-registry-mirror:v0.8.3   "/bin/registry"          6 minutes ago   Up 6 minutes (healthy)
kind-registry-mirror
8e54fe7324ec   kindest/node:v1.36.4                  "/usr/local/bin/entr..."  6 minutes ago   Up 6 minutes   127.0.0.1:57648->6443/tcp
desktop-control-plane
```

Part B — Enable Kubernetes

In a recent Docker Desktop version: go to Docker Desktop → Kubernetes → Create cluster. Docker Desktop supports local clusters using kubeadm or kind — for this practical, a single-node kubeadm cluster is enough.

In the older interface, instead go to Settings → Kubernetes → Enable Kubernetes, then let Docker Desktop start the cluster.

Wait until Docker Desktop shows the cluster as running before continuing.



Part C — Check Kubernetes

Step C1: Check the kubectl version

Run:

```
kubectl version
```

```
C:\>kubectl version
Client Version: v1.34.1
Kustomize Version: v5.7.1
Server Version: v1.36.4
Warning: version difference between client (1.34) and server (1.36) exceeds the supported minor version skew of +/-1
C:\>|
```

Step C2: Check cluster nodes

The important word to look for is Ready.

Run:

```
kubectl get nodes
```

Expected output:

NAME	STATUS	ROLES	AGE	VERSION
docker-desktop	Ready	control-plane	...	v1.xx.x

```
C:\>kubectl get nodes
NAME                                STATUS    ROLES    control-plane   AGE    VERSION
desktop-control-plane              Ready    control-plane   8m10s    v1.36.4
```

If you see Ready, congratulations — you have your first working Kubernetes cluster.

Part D — Understand what you just created

Before doing the practical, understand the layers involved:

```

Your Computer
  |
  v
Docker Desktop
  |
  v
Kubernetes Cluster
  |
  v
Node
  |
  v
Pod
  |
  v
Nginx Container

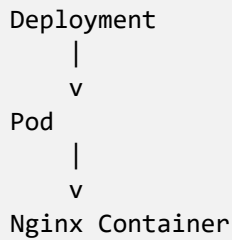
```

Docker vs. Kubernetes

Docker — you directly start a container:

```
docker run nginx
```

Kubernetes — you declare intent, and Kubernetes creates and manages the Pods/containers for you:



A Kubernetes Deployment manages Pods and can restart them if necessary; Deployments are also used for scaling applications.

Part E — Download the Nginx Docker image

Step E1: Pull the Nginx image

We'll use Docker directly first, before bringing Kubernetes in.

Run:

```
docker pull nginx
```

```

C:\>docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
ab606a349520: Pull complete
0478569e858e: Pull complete
3fe5ab3f8614: Pull complete
9302921ce9b3: Pull complete
6310eb16bf42: Pull complete
c06193164a25: Pull complete
76225461b7d3: Pull complete
d18f12d40fc4: Download complete
59379d3e12b2: Download complete
Digest: sha256:cb29e33d254e8b72164ad5b8b9014f44e98e6e5f6d51ef02ee859f96708c90fe
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
  
```

Step E2: Confirm the image was downloaded

Run:

```
docker images
```

Expected output:

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
nginx	latest	xxxxxxxxxxxx	...	...

```
C:\>docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
alpine:latest	28bd5fe8b56d	13MB	3.93MB	
attendance-app:latest	55cf516aafbd	211MB	51.1MB	
attendance-system-app:latest	8297ab74429f	323MB	78.1MB	
attendance-system-backend:latest	3cf379e69b0e	433MB	87.8MB	
attendance-system-frontend:latest	7b6e585be75b	441MB	94.4MB	
attendance-system:latest	d7c42dcd6b0b	310MB	76.9MB	
attendance:latest	6191be07508c	310MB	76.9MB	
college_event-app:latest	fa5279a6cc62	259MB	63MB	
curlimages/curl:latest	58adaa4e8dca	35.4MB	10.7MB	
docker/desktop-cloud-provider-kind:v0.5.0	4ad59ce20658	595MB	162MB	U
docker/desktop-containerd-registry-mirror:v0.0.3	179a9ea3ae47	50.3MB	16.1MB	U
envoyproxy/envoy:v1.36.4	08acc53db6f4	248MB	62.1MB	
html-css-calculator:latest	bdec59b8e5e4	92.5MB	26MB	
jenkins/jenkins:lts	c1e4c349365f	814MB	293MB	U
kindest/node:v1.34.3	08497ee19eac	1.35GB	395MB	
kindest/node:v1.36.4	099e049362a1	1.32GB	385MB	U
kindest/node:v1.37.0-rc.1	aabfbaf916de	1.34GB	388MB	
mongo:7	9bdaeb6dac6e	1.19GB	297MB	
mongo:7.0	9bdaeb6dac6e	1.19GB	297MB	
mongo:latest	e0ce8c35124d	1.3GB	339MB	
nginx:latest	cb29e33d254e	253MB	69.2MB	U
node:22-alpine	c610fcdffb1d5	232MB	58.1MB	
redis:7-alpine	e7723ff73d06	57.8MB	16.8MB	
redis:7-alpine	654e09d4012f	203MB	49.7MB	
simpleapp:v1	6f64b20c3349	240MB	60.3MB	
student_attendance-backend:latest	776a4f2740ab	74MB	21.1MB	
student_attendance-frontend:latest	341fc1057b0e	327MB	80.3MB	
ubuntu:latest	678c6550cc43	160MB	45.3MB	

Part F — Run Nginx using Docker

Step F1: Run the Nginx container

Understand the command before running it:

docker run — create a container

-d — run in the background

--name my-nginx — name the container my-nginx

-p 8080:80 — map computer port 8080 to container port 80

nginx — the Docker image to use

Run:

```
docker run -d --name my-nginx -p 8080:80 nginx
```

```
C:\>docker run -d --name my-nginx -p 8080:80 nginx
5df3720c5931af7a4d3eef2d560c93124741b74933655fb28797c66d25f11f54
```

Step F2: Confirm the container is running

Run:

```
docker ps
```

Expected output:

```
C:\>docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
5df3720c5931   nginx                               "/docker-entrypoint..." 3 minutes ago  Up 3 minutes  0.0.0.0:8080->80/tcp, [:::
f11dd2fd14f8   my-nginx                           "/bin/cloud-provider..." 19 hours ago  Up 19 hours   2375-2376/tcp
4651c2109cad   docker/desktop-cloud-provider-kind:v0.5.0 "/bin/cloud-provider..." 19 hours ago  Up 19 hours   2375-2376/tcp
kind-cloud-provider
kind-registry-mirror
kind-registry-mirror
kind-registry-mirror
kindest/node:v1.36.4 "/bin/registry"         19 hours ago  Up 19 hours (healthy)
8e54fe7324ec   desktop-control-plane              "/usr/local/bin/entr..." 19 hours ago  Up 19 hours   127.0.0.1:57648->6443/tcp
```

Step F3: View Nginx in the browser

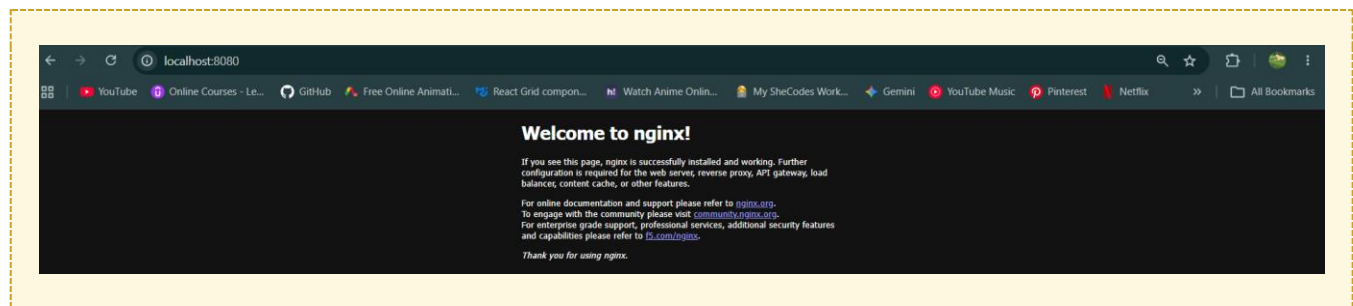
Open your browser and go to the URL below. You should see the default Nginx welcome page.

Run:

```
http://localhost:8080
```

Expected output:

```
Welcome to nginx!
```



You have just deployed Nginx using Docker.

Part G — Stop the Docker container

We don't need this Docker container anymore because Kubernetes will run Nginx from here on.

Step G1: Stop the container

Run:

```
docker stop my-nginx
```

Step G2: Remove the container

Run:

```
docker rm my-nginx
```

Step G3: Confirm it's gone

Run:

```
docker ps
```

```
C:\>docker stop my-nginx
my-nginx
C:\>
```

Expected output:

```
(my-nginx should no longer appear)
```

Part H — Deploy Nginx using Kubernetes

This is the important part of the practical.

Step H1: Create the Deployment

Run:

```
kubectl create deployment nginx --image=nginx
```

Expected output:

```
deployment.apps/nginx created
```

```
C:\>kubectl create deployment nginx --image=nginx
deployment.apps/nginx created
```

You have now created your first Kubernetes Deployment.

Part I — Check the Deployment

Step I1: List Deployments

This means Kubernetes wants 1 Pod, and currently has 1 Pod running.

Run:

```
kubectl get deployments
```

Expected output:

```
C:\>kubectl get deployments
NAME      READY   UP-TO-DATE   AVAILABLE   AGE
nginx     1/1     1            1           29s
```

Part J — Check the Pod

Step J1: List Pods

The random-looking name (e.g. nginx-7c79c4bf97-abcde) is normal — Kubernetes generated it. Don't worry about memorizing it.

Run:

```
kubectl get pods
```

Expected output:

NAME	READY	STATUS	RESTARTS	AGE
nginx-xxxxxxxxxx-xxxxx	1/1	Running	0	30s

```
C:\>kubectl get pods
NAME                        READY   STATUS    RESTARTS   AGE
nginx-7f8fbb96d-2trc2      1/1     Running   0           64s
```

Part K — Understand Deployment and Pod

You now have the following chain of objects:

```

Kubernetes
  |
  v
Deployment: nginx
  |
  v
Pod
  |
  v

```

<pre> Nginx Container v Port 80 </pre>	
Object	Meaning
Deployment	Controls the desired number of Pods.
Pod	The basic execution unit in Kubernetes.
Container	Your actual application runs inside the container.

Part L — See the Pod details

Step L1: Describe the Pod

Replace <POD_NAME> with the name you saw in Part J, e.g. nginx-7c79c4bf97-abcde. Look for the Containers: section in the output.

Run:

```

kubectl describe pod <POD_NAME>

# example:
kubectl describe pod nginx-7c79c4bf97-abcde

```

```
C:\>kubectl describe pod nginx-7f8fbb96d-2trc2
Name:          nginx-7f8fbb96d-2trc2
Namespace:     default
Priority:       0
Service Account: default
Node:          desktop-control-plane/172.18.0.2
Start Time:    Fri, 18 Sep 2026 16:04:42 +0530
Labels:        app=nginx
               pod-template-hash=7f8fbb96d
Annotations:   <none>
Status:        Running
IP:            10.244.0.5
IPs:           10.244.0.5
Controlled By: ReplicaSet/nginx-7f8fbb96d
Containers:
  nginx:
    Container ID:  containerd://db57fc3e3358b6136d216f3fed1b9bf65207b2d6517bcaa8d65efe0099de7fac
    Image:         nginx
    Image ID:      docker.io/library/nginx@sha256:cb29e33d254e8b72164ad5b8b9014f44e98e6e5fed51ef02ee859f9c708c90fe
    Port:          <none>
    Host Port:     <none>
    State:         Running
      Started:     Fri, 18 Sep 2026 16:04:55 +0530
    Ready:         True
    Restart Count: 0
    Environment:   <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-vzpgq (ro)
Conditions:
  Type              Status
  PodReadyToStartContainers  True
  Initialized        True
  Ready              True
  ContainersReady    True
  PodScheduled       True
Volumes:
  kube-api-access-vzpgq:
    Type:              Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds: 3607
    ConfigMapName:      kube-root-ca.crt
    Optional:           false
    DownwardAPI:        true
    QoS Class:          BestEffort
    Node-Selectors:     <none>
    Tolerations:        node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                       node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
  Type    Reason      Age   From          Message
  ----    -
  Normal  Scheduled   2m27s default-scheduler  Successfully assigned default/nginx-7f8fbb96d-2trc2 to desktop-control-plane
  Normal  Pulling     2m26s kubelet        Pulling image "nginx"
  Normal  Pulled      2m19s kubelet        Successfully pulled image "nginx" in 12.213s (12.213s including waiting). Image size: 66343686 bytes.
  Normal  Created     2m14s kubelet        Container created
  Normal  Started     2m14s kubelet        Container started
```

Step L2: Get extended Pod info

Run:

```
kubectl get pods -o wide
```

```
C:\>kubectl get pods -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP            NODE                NOMINATED NODE   READINESS GATES
nginx-7f8fbb96d-2trc2              1/1     Running   0           3m34s  10.244.0.5    desktop-control-plane  <none>           <none>
```

Part M — Check Nginx logs

Step M1: View Pod logs

Run:

```
kubectl logs <POD_NAME>
```

example:

```
C:\>kubectl logs nginx-7f8fbb96d-2trc2
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/09/18 10:34:55 [notice] 1#1: using the "epoll" event method
2026/09/18 10:34:55 [notice] 1#1: nginx/1.31.6
2026/09/18 10:34:55 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/09/18 10:34:55 [notice] 1#1: OS: Linux 6.6.87.2-microsoft-standard-WSL2
2026/09/18 10:34:55 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1073741816:1073741816
2026/09/18 10:34:55 [notice] 1#1: start worker processes
2026/09/18 10:34:55 [notice] 1#1: start worker process 33
2026/09/18 10:34:55 [notice] 1#1: start worker process 34
2026/09/18 10:34:55 [notice] 1#1: start worker process 35
2026/09/18 10:34:55 [notice] 1#1: start worker process 36
```

This is the Kubernetes equivalent of the Docker command:

```
docker logs my-nginx      (Docker)
kubectl logs <POD_NAME>  (Kubernetes)
```

Part N — Our application is NOT accessible yet

You created Deployment → Pod → Nginx, but if you try `http://localhost` you should not expect your Kubernetes Nginx to automatically appear there.

Why? Because the Pod only has an internal Kubernetes network address. We need a Service to expose it.

Part O — Create a Kubernetes Service

Step O1: Expose the Deployment

Run:

```
kubectl expose deployment nginx --type=NodePort --port=80
```

Expected output:

```
C:\>kubectl expose deployment nginx --type=NodePort --port=80
service/nginx exposed
```

Step O2: List Services

The important part is the port mapping, e.g. `80:30080/TCP`. Your exact NodePort will differ.

Run:

```
kubectl get services
```

Expected output:

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
kubernetes	ClusterIP	10.x.x.x	<none>	443/TCP
nginx	NodePort	10.x.x.x	<none>	80:30xxx/TCP

```
C:\>kubectl get services
NAME          TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
kubernetes    ClusterIP     10.96.0.1     <none>         443/TCP          19h
nginx         NodePort      10.96.207.238 <none>         80:30710/TCP     21s
```

Part P — Access Nginx

Because you're using Docker Desktop Kubernetes, exactly how the Service is reachable can depend on your cluster setup.

Step P1: Check the Service

Run:

```
kubectl get service nginx
```

```
C:\>kubectl get service nginx
NAME          TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
nginx         NodePort      10.96.207.238 <none>         80:30710/TCP     58s
```

Step P2: Port-forward to the Service

This is the most reliable way to reach the Service from Docker Desktop. Keep this terminal running.

Run:

```
kubectl port-forward service/nginx 8080:80
```

Expected output:

```
C:\>kubectl port-forward service/nginx 8080:80
Forwarding from 127.0.0.1:8080 -> 80
Forwarding from [::1]:8080 -> 80
```

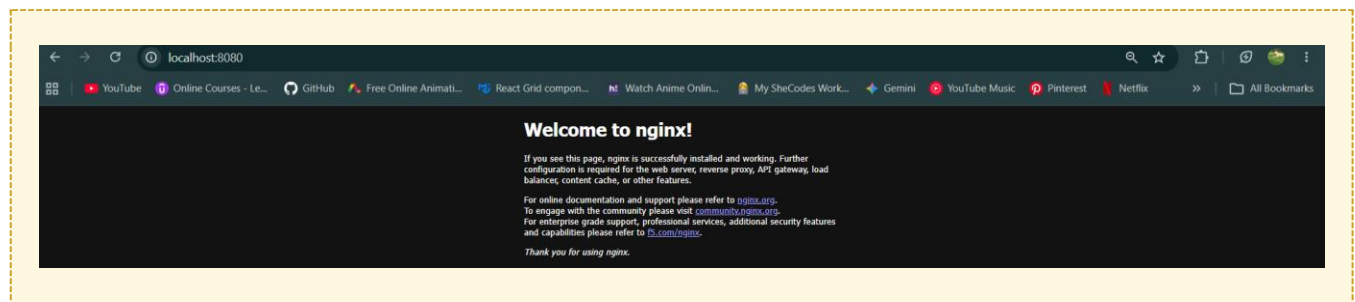
Step P3: Open Nginx in the browser

Run:

```
http://localhost:8080
```

Expected output:

```
Welcome to nginx!
```



You have now completed the full chain:

```

  Docker Image
    |
    v
Kubernetes Deployment
    |
    v
  Pod
    |
    v
Nginx Container
    |
    v
Kubernetes Service
    |
    v
Your Browser
  
```

Part Q — The most important experiment: scaling

Currently you have one Pod behind the Deployment:

```

nginx Deployment
  |
  v
  Pod 1
  
```

Step Q1: Scale the Deployment to 3 replicas

Run:

```
kubectl scale deployment nginx --replicas=3
```

Expected output:

```
C:\>kubectl scale deployment nginx --replicas=3
deployment.apps/nginx scaled
```

Step Q2: Confirm 3 Pods are running

Run:

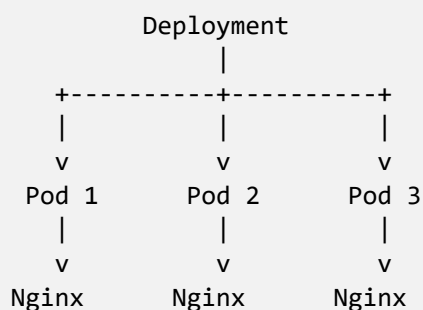
```
kubectl get pods
```

Expected output:

NAME	READY	STATUS	RESTARTS
nginx-xxxxx-aaaaa	1/1	Running	0
nginx-xxxxx-bbbbb	1/1	Running	0
nginx-xxxxx-cccc	1/1	Running	0

```
C:\>kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
nginx-7f8fbb96d-2trc2              1/1     Running   0           9m20s
nginx-7f8fbb96d-br2f9              1/1     Running   0           18s
nginx-7f8fbb96d-nt6xt              1/1     Running   0           18s
```

You now have:



This is one of the fundamental ideas behind Kubernetes.

Part R — Check the Deployment again

Step R1: List the Deployment

3/3 means three Pods are ready out of three desired Pods.

Run:

```
kubectl get deployment
```

Expected output:

```
C:\>kubectl get deployment
NAME      READY   UP-TO-DATE   AVAILABLE   AGE
nginx     3/3     3             3           10m
```

Part S — Test Kubernetes self-healing

Step S1: List Pods and copy one Pod name

Run:

```
kubectl get pods
```

Expected output:

```
nginx-7f8fbb96d-2trc2
```

Step S2: Delete that Pod

You might think your application is now down.

Run:

```
kubectl delete pod nginx-7f8fbb96d-2trc2
```

```
C:\>kubectl delete pod nginx-7f8fbb96d-2trc2
pod "nginx-7f8fbb96d-2trc2" deleted from default namespace
```

Step S3: Immediately check Pods again

You may see a new Pod already being created.

Run:

```
kubectl get pods
```

```
C:\>kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
nginx-7f8fbb96d-br2f9	1/1	Running	0	3m58s
nginx-7f8fbb96d-n8p49	1/1	Running	0	77s
nginx-7f8fbb96d-nt6xt	1/1	Running	0	3m58s

Step S4: Check again after a few seconds

You should eventually be back to 3 Pods. Because you told Kubernetes "I want 3 replicas", when it sees Desired = 3 but Actual = 2, it creates another Pod. This is the beginning of self-healing.

Run:

```
kubectl get pods
```

Expected output:

```
(3 Pods Running again)
```

```
C:\>kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
nginx-7f8fbb96d-br2f9	1/1	Running	0	3m58s
nginx-7f8fbb96d-n8p49	1/1	Running	0	77s
nginx-7f8fbb96d-nt6xt	1/1	Running	0	3m58s

Part T — View everything together

Step T1: List all resources

This one command is very useful for beginners — it shows Pods, Services, Deployments, and ReplicaSets in one place.

Run:

```
kubectl get all
```

```
C:\>kubectl get all
NAME                                READY   STATUS    RESTARTS   AGE
pod/nginx-7f8fbb96d-br2f9          1/1     Running   0           4m47s
pod/nginx-7f8fbb96d-n8p49          1/1     Running   0           2m6s
pod/nginx-7f8fbb96d-nt6xt          1/1     Running   0           4m47s

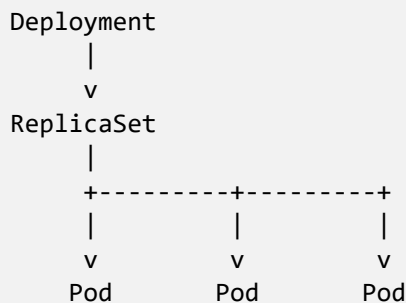
NAME                                TYPE               CLUSTER-IP    EXTERNAL-IP   PORT(S)          AGE
service/kubernetes                 ClusterIP         10.96.0.1     <none>        443/TCP          19h
service/nginx                      NodePort          10.96.207.238 <none>        80:30710/TCP     8m44s

NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/nginx              3/3     3            3           13m

NAME                                DESIRED   CURRENT   READY   AGE
replicaset.apps/nginx-7f8fbb96d    3         3         3       13m
C:\>|
```

Part U — Understand ReplicaSet

You didn't directly create a ReplicaSet — but when you created the Deployment, Kubernetes created one automatically.



Deployment manages the ReplicaSet, and the ReplicaSet manages the desired number of Pods.

Part V — Inspect the Service

Step V1: Describe the Service

Run:

```
kubectl describe service nginx
```

Expected output:

```
Type: NodePort
Port: 80
```

TargetPort: 80
Endpoints: ...

```
C:\>kubectl describe service nginx
Name: nginx
Namespace: default
Labels: app=nginx
Annotations: <none>
Selector: app=nginx
Type: NodePort
IP Family Policy: SingleStack
IP Families: IPv4
IP: 10.96.207.238
IPs: 10.96.207.238
Port: <unset> 80/TCP
TargetPort: 80/TCP
NodePort: <unset> 30710/TCP
Endpoints: 10.244.0.7:80,10.244.0.6:80,10.244.0.8:80
Session Affinity: None
External Traffic Policy: Cluster
Internal Traffic Policy: Cluster
Events: <none>
```

Think of the traffic flow as:

```
Service
|
| port 80
v
Pods
|
| port 80
v
Nginx
```

Part W — Clean up

When your practical is complete, remove the resources you created.

Step W1: Delete the Service

Run:

```
kubectl delete service nginx
```

Step W2: Delete the Deployment

Run:

```
kubectl delete deployment nginx
```

Step W3: Confirm everything is gone

Run:

```
kubectl get all
```

Expected output:

(your nginx resources should no longer appear)

```
C:\>kubectl get all
NAME                TYPE          CLUSTER-IP   EXTERNAL-IP   PORT(S)    AGE
service/kubernetes  ClusterIP     10.96.0.1    <none>        443/TCP    19h
```

What you should understand after Practical 1

Concept	Simple meaning
Docker Image	Blueprint for a container
Container	Running instance of an image
Kubernetes Cluster	Kubernetes environment
Node	Machine running Kubernetes workloads
Pod	Smallest deployable Kubernetes unit
Deployment	Manages application Pods
ReplicaSet	Maintains desired number of Pods
Service	Provides stable network access to Pods
NodePort	Exposes a Service through a node port
Scaling	Increasing/decreasing number of Pods
Self-healing	Kubernetes recreates failed/deleted Pods

Appendix — Full command list (lab notebook copy)

```

docker --version
docker pull nginx
docker run -d --name my-nginx -p 8080:80 nginx
docker ps
docker stop my-nginx
docker rm my-nginx
kubectl get nodes
kubectl create deployment nginx --image=nginx
kubectl get deployments
kubectl get pods
kubectl get pods -o wide
kubectl describe pod <POD_NAME>
kubectl logs <POD_NAME>
kubectl expose deployment nginx --type=NodePort --port=80
kubectl get services
kubectl port-forward service/nginx 8080:80
kubectl scale deployment nginx --replicas=3
kubectl get pods
kubectl get deployment
kubectl delete pod <POD_NAME>
kubectl get pods
kubectl get all
kubectl delete service nginx
kubectl delete deployment nginx

```

